

Real-Time Function Hooking – Frida Guide

Frida ke saath real-time function hooking ka matlab hai application ke runtime mein uske methods ya functions ke behavior ko intercept karna aur modify karna. Yeh dynamic analysis ka sab se powerful feature hai .

Java Layer Hooking (Android)

Android apps ke Java/Kotlin code ko hook karne ke liye Frida ka **Java API** use hota hai. Sab se pehle `Java.perform()` ke andar code likhna zaroori hai taake Java Virtual Machine (VM) available ho .

1. `Java.use()` – Class Access Karna

Pehle target class ka reference lena hota hai:

```
Java.perform(function () {  
    var Activity =  
    Java.use("android.app.Activity");  
    // ab Activity class ke methods ko hook kar  
    sakte hain  
});
```

Java.use() method se aap kisi bhi Java class ko access kar sakte hain aur uske methods ko manipulate kar sakte hain .

2. implementation – Method Override Karna

Method ke original behavior ko override karne ke liye implementation property use hoti hai:

```
Java.perform(function () {  
    var Activity =  
    Java.use("android.app.Activity");  
    Activity.onResume.implementation = function ()  
{  
        console.log("[*] onResume() got called!");  
        this.onResume(); // original method call  
    };  
});
```

Is script mein onResume method ko hook kiya gaya hai. Jab bhi koi activity resume hogi, pehle console mein message print hoga, phir original method call hogi .

3. Arguments aur Return Value Log Karna

JADX-GUI se Frida snippet copy karne ka feature bhi available hai. Right-click method name → "Copy as frida snippet (f)" select karein :

```
let a = Java.use("sg.vantagepoint.a.a");  
a["a"].implementation = function (bArr, bArr2) {  
    console.log('a is called' + ', ' + 'bArr: ' +
```

```

bArr + ', ' + 'bArr2: ' + bArr2);
    let ret = this.a(bArr, bArr2);
    console.log('a ret value is ' + ret);
    return ret;
};

```

Yeh script `sg.vantagepoint.a.a` class ke a method ko hook karti hai aur uske arguments aur return value ko console mein log karti hai .

4. Heap Search – Live Instances Find Karna

Frida se Java heap par live objects bhi search kar sakte hain:

```

setImmediate(function() {
    console.log("[*] Starting script");
    Java.perform(function () {
        Java.choose("android.view.View", {
            "onMatch":function(instance){
                console.log("[*] Instance found:
" + instance.toString());
            },
            "onComplete":function() {
                console.log("[*] Finished heap
search")
            }
        });
    });
});

```

Yeh script saare android.view.View objects ke live instances dhundhti hai aur unka toString() result print karti hai .

Native Layer Hooking (C/C++ Libraries)

Native functions (.so libraries mein) ko hook karne ke liye Frida ka **Interceptor API** use hota hai .

1. Exported Function Hook Karna

Export table mein available functions ko name se hook karna:

```
Interceptor.attach(
```

```
Process.getModuleByName('libc.so').getExportByName('open'),
{
    onEnter: function(args) {
        const path = args[0].readCString();
        console.log("[*] open() called with
path: " + path);
    },
    onLeave: function(retval) {
        console.log("[*] open() returned: " +
retval);
    }
}
```

```
);
```

Yeh script libc.so ke open function ko hook karti hai. onEnter function entry par call hota hai, onLeave function exit par .

2. Unexported Function Hook Karna (Offset Based)

Unexported functions (jo export table mein nahi hain) ko hook karne ke liye base address + offset use karte hain:

```
Java.perform(function () {
    var base =
Module.findBaseAddress("libtarget.so");
    var targetAddr = base.add(0x5678); //
IDA/Ghidra se nikala gaya offset
    Interceptor.attach(targetAddr, {
        onEnter: function (args) {
            console.log("[*] Parameter 1: " +
args[0].toInt32());
        },
        onLeave: function (retval) {
            retval.replace(999); // return value
modify
        }
    });
});
```

Important: ARM32 architecture par Thumb instructions ke liye offset mein +1 add karna padta hai (base + offset + 1) .

frida-trace – Quick Function Tracing

frida-trace ek built-in tool hai jo bohot saare functions ko ek saath trace karne ke liye use hota hai .

Basic Usage

Spawn mode (app launch ke saath):

```
frida-trace -U -f com.example.app -j  
'javax.crypto.Cipher!$init'
```

Attach mode (already running app):

```
frida-trace -U -p 12345 -j  
'javax.crypto.Cipher!$init'
```

Wildcard Hooking – Thousands of Functions

* wildcard se multiple classes aur methods ko ek saath hook kiya ja sakta hai :

All JSON-related methods:

```
frida-trace -U -p 12345 -j '*json*!*' -j  
'*JSON*!*'
```

All SQL-related methods:

```
frida-trace -U -p 12345 -j '*SQL*!*' -j '*sql*!*'
```

Exclude karna (spammy methods):

```
frida-trace -U -p 12345 -j '*SQL*!*' -j '*sql*!*'  
-J '*ProblemSQLClass*!*
```

iOS mein Objective-C Tracing

iOS par -m flag se Objective-C methods trace kar sakte hain :

```
frida-trace -U YourApp -m "[NSURL* *HTTP*]"
```

Yeh command un saari methods ko trace karegi jinka name "HTTP" contain karta hai aur jo NSURL prefix wali class mein hain .

Advanced: Dynamically Loaded Classes Hook Karna

Java.use() sirf already loaded classes par kaam karti hai. Agar class runtime par load ho rahi hai toh ClassLoader.loadClass() hook karke use handle kiya ja sakta hai :

```
JavaUseOnceLoaded("com.example.DynamicClass",  
function(clazz) {  
    console.log("[*] Class loaded: " +  
clazz.$className);
```

```
// ab clazz ke methods hook kar sakte hain  
});
```

Important Points

- `Java.perform()` – Hamesha Java code iske andar likhein taake Java VM available ho
- **Version matching:** PC aur phone par Frida versions match honi chahiye
- **No guarantee:** Kuch functions (suspend functions, coroutines) hook nahi ho pati

 **Educational Purpose Only:** Function hooking sirf apne own apps aur authorized testing environments mein karein. Kisi doosre ke app mein bina permission ke hooking karna applicable cyber laws ke under punishable hai.

MADE BY:
SYED GHOUS
